阶段 0：统一书籍数据结构

Eternity-Library 数据库线程化重构的第一步

本教程不进行线程重构，也不改变数据库读写职责。我们先解决一个更基础、也更容易被忽略的问题：数据库返回的是元组，表格模型把它变成列表，封面模型又按元组下标读取，多个模块对“一本书长什么样”没有统一约定。

> **本次操作范围：**你将手动新增 `book_record.py`，然后手动修改两个 Model 和部分查询代码。本文只提供步骤和可运行代码，我不会直接修改项目源码。

适合对象：非计算机专业、正在通过项目学习 Python/PySide6 的业余开发者

本阶段目标：所有模块使用同一种 `BookRecord` 表示书籍；数据库列名、列顺序和中文表头集中管理。

完成标准：应用可以启动，已有查询和两个视图仍能显示，单独测试脚本可以读取并打印书籍数据。

1. 阶段 0 到底要解决什么

目前数据库表有 19 个字段：

位置	数据库字段	含义
0	`book_id`	书籍 ID
1	`book_name`	书名
2	`book_path`	本地路径
3	`add_time`	加入时间
4	`author`	作者
5	`nationality`	作者国籍
6	`translator`	译者
7	`publisher`	出版社
8	`publication_date`	出版日期
9	`level`	书籍等级
10	`read_status`	阅读状态
11	`book_type`	书籍类型
12	`isbn`	ISBN
13	`pages`	页数
14	`read_progress`	阅读进度
15	`read_time`	阅读时间
16	`read_date`	阅读日期
17	`read_link`	阅读链接
18	`introduction`	简介

现在不同代码使用不同方式表示一本书：

- `BookQueryWorker` 查询结果：`list[tuple]`。
- `BookTableModel`：当前代码会把每个元组复制成 `list`，用于编辑单元格。
- `CoverCardModel`：把元组转换成只包含少数字段的 `dict`。
- `DatabaseManager` 写入时：使用任意关键字参数字典。

阶段 0 完成后的数据流



> **为什么先做这个：**如果不先统一数据结构，后面把查询移动到工作线程时，信号传回来的数据格式仍然混乱；线程问题、模型问题和字段下标问题会同时出现，排错会非常困难。

2. 设计决定：新增一个简单的数据结构模块

我们新增文件 `book_record.py`，里面只放四类内容：

1. 数据库字段名的固定顺序。
2. 字段对应的中文表头。
3. `BookRecord` 数据类。
4. 把 SQLite 查询行转换为 `BookRecord` 的函数。

这里使用 Python 的 `dataclass`。它可以理解为“自动帮我们生成初始化方法的数据容器”。它不是数据库，也不是线程，也不会自动做复杂事情，适合当前的学习阶段。

> **为什么不继续使用数字下标：**`book[11]` 只有作者记得第 11 列是什么时才安全。一旦数据库增加列、查询列顺序改变，程序可能不报错但显示错误数据。使用 `book.book_type` 更容易阅读和检查。

3. 第一步：新增 book_record.py

操作 1：在项目根目录新建文件

在 `D:\Eternity-Library` 下新建文件，文件名必须是：

```
book_record.py
```

将下面的完整代码复制进去并保存。这个文件是可以直接运行的 Python 代码，不是伪代码。

```python
from __future__ import annotations

from dataclasses import dataclass, fields
from typing import Any, Iterable, Sequence


# 这个顺序必须与 database.py 中 CREATE TABLE 的字段顺序一致。
BOOK_COLUMNS: tuple[str, ...] = (
    "book_id",
    "book_name",
    "book_path",
    "add_time",
    "author",
    "nationality",
    "translator",
    "publisher",
    "publication_date",
    "level",
    "read_status",
    "book_type",
    "isbn",
    "pages",
    "read_progress",
    "read_time",
    "read_date",
    "read_link",
    "introduction",
)


# 列名和中文表头必须一一对应，长度也必须相同。
BOOK_COLUMN_TITLES: tuple[str, ...] = (
    "书籍ID",
    "书籍名称",
    "书籍路径",
    "添加时间",
    "作者",
    "国籍",
    "译者",
    "出版社",
    "出版日期",
    "书籍等级",
    "阅读状态",
    "书籍类型",
    "ISBN",
    "页数",
    "阅读进度",
    "阅读时间",
    "阅读日期",
    "阅读链接",
    "简介",
)


if len(BOOK_COLUMNS) != len(BOOK_COLUMN_TITLES):
    raise RuntimeError("书籍字段数量与中文表头数量不一致")
```

```python
@dataclass
class BookRecord:
    """表示数据库中的一条书籍记录。"""

    book_id: int | None = None
    book_name: str = ""
    book_path: str | None = None
    add_time: str | None = None
    author: str | None = None
    nationality: str | None = None
    translator: str | None = None
    publisher: str | None = None
    publication_date: str | None = None
    level: str | None = None
    read_status: str | None = None
    book_type: str | None = None
    isbn: int | None = None
    pages: int | None = None
    read_progress: str | None = None
    read_time: float | None = None
    read_date: str | None = None
    read_link: str | None = None
    introduction: str | None = None

    @classmethod
    def from_row(cls, row: Sequence[Any]) -> "BookRecord":
        """把 SQLite 返回的一行数据转换成 BookRecord。"""
        if len(row) != len(BOOK_COLUMNS):
            raise ValueError(
                f"书籍记录应有 {len(BOOK_COLUMNS)} 个字段，实际收到 {len(row)} 个"
            )
        return cls(*row)

    @classmethod
    def from_mapping(cls, values: dict[str, Any]) -> "BookRecord":
        """从字段字典创建 BookRecord，适合扫描结果转成统一对象。"""
        unknown_keys = set(values) - set(BOOK_COLUMNS)
        if unknown_keys:
            names = ", ".join(sorted(unknown_keys))
            raise ValueError(f"发现未知书籍字段：{names}")

        allowed_values = {
            field.name: values[field.name]
            for field in fields(cls)
            if field.name in values
        }
        return cls(**allowed_values)

    def to_row(self) -> tuple[Any, ...]:
        """按数据库字段顺序转换为可用于 INSERT 的元组。"""
        return tuple(getattr(self, column) for column in BOOK_COLUMNS)

    def to_mapping(self) -> dict[str, Any]:
        """转换为字段名到字段值的字典。"""
        return {
```

```
            column: getattr(self, column)
            for column in BOOK_COLUMNS
        }


def rows_to_records(rows: Iterable[Sequence[Any]]) -> list[BookRecord]:
    """把多行 SQLite 查询结果转换成 BookRecord 列表。"""
    return [BookRecord.from_row(row) for row in rows]
```

代码逐段解释

代码	作用
`BOOK_COLUMNS`	集中记录字段顺序，后续 SQL、表头和数据转换都可以引用它。
`BookRecord`	一条书籍记录；每个属性对应数据库中的一列。
`from_row()`	把 SQLite 的元组转换成有名字的对象，并检查字段数量。
`from_mapping()`	把扫描模块产生的字典转换成统一对象，并拒绝未知字段。
`to_row()`	写入数据库时按固定顺序生成元组。
`to_mapping()`	需要动态生成 INSERT 列名时，转换回字典。

> **操作 2：先单独运行这个文件**
>
> 打开 PowerShell，进入项目目录，执行：
>
> ```
> cd D:\Eternity-Library
> python -c "from book_record import BookRecord, BOOK_COLUMNS; book = BookRecord(book_name='测试书'); print(book.book_name); print(len(BOOK_COLUMNS)); print(book.to_mapping())"
> ```
>
> 预期结果中应出现 `测试书`、数字 `19`，以及一个包含 19 个字段的字典。没有错误信息就说明基础数据结构可以导入。

4. 第二步：为数据库查询增加统一转换

这一小步仍然不涉及线程。先让现有的 `BookQueryWorker` 在查询完成后把元组转换为 `BookRecord`。这样之后的 Model 和新线程 Worker 都可以使用同一种数据。

操作 3：修改 database.py 的 import 区域

打开 `database.py`，在现有 import 后面加入：

```
from book_record import (
    BOOK_COLUMNS,
    BOOK_COLUMN_TITLES,
    BookRecord,
    rows_to_records,
)
```

如果编辑器提示导入未使用，暂时不用担心；接下来几步会使用这些名称。

操作 4：替换 BookQueryWorker 的类型和转换部分

在 `BookQueryWorker` 中，将原来的 `_execute_query` 方法整体替换为下面代码：

```
def _execute_query(
    self,
    sql: str,
    params: tuple = (),
) -> list[tuple] | None:
    """
    在工作线程中执行查询，返回 SQLite 原始行。
    查询失败时返回 None，具体错误由 query_error 发出。
    """
    conn = sqlite3.connect(self.db_name)
    try:
        cursor = conn.cursor()
        cursor.execute(sql, params)
        result = cursor.fetchall()
        cursor.close()
        return result
    except sqlite3.Error as error:
        self.query_error.emit(str(error))
        return None
    finally:
        conn.close()
```

然后，把 `fetch_all_books` 整个方法替换为：

```
@Slot()
def fetch_all_books(self):
    """查询全部书籍，并把每一行转换为 BookRecord。"""
    result = self._execute_query(
        "SELECT * FROM books_information ORDER BY book_id"
    )
    if result is None:
        return

    try:
        books = rows_to_records(result)
    except (TypeError, ValueError) as error:
        self.query_error.emit(f"书籍数据格式错误：{error}")
        return

    self.books_ready.emit(books)
```

同时，把类顶部的信号注释调整为以下形式：

```
books_ready = Signal(list)      # list[BookRecord]
query_error = Signal(str)
trigger_fetch = Signal()
```

> **为什么把错误改成 None：**原代码允许 `_execute_query()` 返回字符串错误信息，但调用方只判断“是不是 None”。这样字符串可能被当作正常列表传给界面。现在错误立即通过 `query_error` 发出，成功结果永远是记录列表，含义更清楚。

处理空数据库

注意：空数据库查询成功时，`result` 是空列表，不是 `None`。所以必须继续发出 `books_ready([])`。界面层以后应该清空 Model，而不是直接 `if not books: return` 保留旧画面。

5. 第三步：让表格模型使用 BookRecord

你当前的 `table_model.py` 已经有未提交修改：它会把查询元组复制成列表，以便编辑单元格。这个修改不要覆盖。阶段 0 需要做的是，在保留编辑能力的前提下，明确每一行是 `BookRecord` 还是可编辑列表。

为了不在本阶段同时设计“编辑后保存数据库”，这里采用一个容易理解的规则：

- 数据库查询结果在进入 Model 前是 `BookRecord`。
- 表格 Model 内部仍然保存可编辑的列表，因为当前 `setData()` 需要修改单元格。
- 列表由 `BookRecord.to_row()` 生成，不再依赖查询结果本身恰好是元组。

操作 5：修改 table_model.py 的 import

在现有 PySide6 import 后加入：

```
from book_record import BOOK_COLUMN_TITLES, BookRecord
```

操作 6：替换 BookTableModel.__init__ 和 set_data

将 `__init__` 改为：

```
def __init__(self, column_titles=None, parent=None):
    super().__init__(parent)
    self._table_books: list[list[object]] = []
    self._column_titles = column_titles or list(BOOK_COLUMN_TITLES)
```

将 `set_data` 改为：

```
def set_data(self, books: list[BookRecord]):
    """把 BookRecord 列表转换成表格内部可编辑的二维列表。"""
    self.beginResetModel()
    self._table_books = [
        list(book.to_row())
        for book in books
    ]
    self.endResetModel()
```

操作 7：检查 data() 中的字段位置

你当前的 `data()` 使用 0、1、4、11 这些位置作为自定义 Role，这些位置与数据库字段顺序一致，可以暂时保留。为了让代码更容易学习，建议把文件顶部增加下面的常量：

```
BOOK_ID_INDEX = 0
BOOK_NAME_INDEX = 1
AUTHOR_INDEX = 4
BOOK_TYPE_INDEX = 11
```

然后把相应代码改成：

```
elif role == self.BookIdRole:
    return self._table_books[row][BOOK_ID_INDEX]
elif role == self.BookNameRole:
    return self._table_books[row][BOOK_NAME_INDEX]
elif role == self.AuthorRole:
    return self._table_books[row][AUTHOR_INDEX]
elif role == self.BookTypeRole:
    return self._table_books[row][BOOK_TYPE_INDEX]
```

这一步不是最终方案。阶段 1 可以进一步改成按属性读取，或者让表格 Model 直接保存 `BookRecord` 并单独管理编辑值。当前目标是先保证数据转换路径清晰且可以运行。

6. 第四步：让封面模型使用 BookRecord

封面模型是目前最明显的“数据格式转换点”：它现在接收元组后使用 `b[0]`、`b[1]`、`b[4]`、`b[11]`。改成属性访问后，读代码时可以直接知道每个值是什么。

操作 8：修改 cover_model.py 的 import

加入：

```
from book_record import BookRecord
```

操作 9：替换 set_books 方法

把 `CoverCardModel.set_books` 整个替换为：

```
def set_books(self, books: list[BookRecord]):
    """
    批量替换全部数据。
    Model 位于主线程，只有主线程收到查询结果后才能调用此方法。
    """
    self.beginResetModel()
    self._books = [
        {
            "id": book.book_id,
            "name": book.book_name or "未知书名",
            "path": book.book_path,
            "author": book.author or "未知作者",
            "type": book.book_type or "未知类型",
            "cover_path": None,
        }
        for book in books
    ]
    self.endResetModel()
```

这样封面模型仍保存适合界面显示的字典，但字典的来源已经明确是 `BookRecord`，而不是依赖数字下标。

7. 第五步：修改 MainWindow 的表头来源

当前 `MainWindow.__init__` 创建了一个临时的 `BookQueryWorker`，并调用同步的 `column_titles_translation()` 获取表头。阶段 0 可以先把表头移到统一模块，避免为了 19 个固定文字额外打开一次数据库连接。

操作 10：修改 el_gui.py 的 import

在 import 区域加入：

```
from book_record import BOOK_COLUMN_TITLES
```

操作 11：修改 MainWindow.__init__ 中的表头初始化

找到：

```
self.db = BookQueryWorker()
self.column_titles = self.db.column_titles_translation()
```

替换为：

```
self.column_titles = list(BOOK_COLUMN_TITLES)
```

暂时不要删除下面的 `self._book_worker = BookQueryWorker()`，因为它仍然是实际执行查询的 Worker。阶段 1 统一读写线程时再处理它。

重要：这一步之后，`BookQueryWorker.column_titles()` 和 `column_titles_translation()` 暂时可以保留，避免一次改太多；但它们已经不再被 MainWindow 使用。等阶段 0 验证完成后，可以在阶段 1 再删除旧接口。

8. 第六步：修改 MainWindow 的结果处理类型

因为 `books_ready` 现在传回 `list[BookRecord]`，`_on_books_loaded` 的注释和空列表处理应该同步调整。

操作 12：替换 _on_books_loaded

在 `el_gui.py` 中，将现有方法替换为：

```
def _on_books_loaded(self, books: list[BookRecord]) -> None:
    """收到后台查询结果，在主线程更新缓存和两个 Model。"""
    self._books_cache = books
    self._cache_timestamp = time.time()

    # 空列表也是一次成功查询，必须清空旧数据显示。
    self.cover_model.set_books(books)
    self.table_model.set_data(books)

    if books:
        message = self._translate(
            "Views",
            f"已刷新，共 {len(books)} 本书",
        )
    else:
        message = self._translate("Views", "数据库中暂无书籍")
    self.statusbar.showMessage(message)
```

同时，在 el_gui.py 的 import 区域加入：

```
from book_record import BookRecord
```

现有缓存变量也会变成 `list[BookRecord] | None`。这不会改变缓存的基本逻辑，只是让缓存中的数据格式明确。

9. 第七步：让扫描数据也可以转换

当前 `books.py` 的扫描结果是字典，随后直接传给 `DatabaseManager.add_book(**i)`。阶段 0 不要求现在就迁移写入线程，但可以先验证扫描字典是否符合统一字段。

操作 13：在 books.py 中增加一次转换测试

先在 `books.py` 的 import 区域加入：

```
from book_record import BookRecord
```

然后在 `write_into_database` 开头暂时加入下面代码：

```
def write_into_database(self, scan_info):
    """将扫描结果写入数据库"""
    records = [
        BookRecord.from_mapping(book_info)
        for book_info in scan_info
    ]

    db = DatabaseManager()
    for record in records:
        db.add_book(**record.to_mapping())
    db.close()
```

这段代码可以直接运行，仍然使用原来的 `DatabaseManager` 写入方式，只是在写入前增加了字段检查和统一转换。

可能遇到的错误是有用的：如果扫描结果字典中有数据库没有的字段，程序会明确提示“发现未知书籍字段”；如果缺少某些字段，`BookRecord` 会使用默认值。这样比后面 SQL 才报错更容易定位。

10. 第八步：增加独立验证脚本

不要每次验证数据结构都启动完整 GUI。新增一个临时或长期保留的小脚本，可以快速检查数据库字段数量、查询结果转换和 Model 所需字段。

操作 14：新建 verify_phase0.py

在项目根目录新建 `verify_phase0.py`，复制下面完整代码：

```
import sqlite3

from book_record import (
    BOOK_COLUMNS,
    BOOK_COLUMN_TITLES,
    BookRecord,
    rows_to_records,
)


DB_NAME = "books_information.db"


def main() -> None:
    print("1. 检查字段定义")
    print(f"字段数量：{len(BOOK_COLUMNS)}")
    print(f"表头数量：{len(BOOK_COLUMN_TITLES)}")
    assert len(BOOK_COLUMNS) == 19
    assert len(BOOK_COLUMNS) == len(BOOK_COLUMN_TITLES)

    print("2. 读取数据库实际字段")
    connection = sqlite3.connect(DB_NAME)
    try:
        rows = connection.execute(
            "SELECT * FROM books_information ORDER BY book_id"
        ).fetchall()
    finally:
        connection.close()

    print(f"数据库记录数量：{len(rows)}")
    if rows:
        assert len(rows[0]) == len(BOOK_COLUMNS)
        records = rows_to_records(rows)
        first = records[0]
        assert isinstance(first, BookRecord)
        print(f"第一本书：{first.book_name}")
        print(f"作者：{first.author or '未填写'}")
        print(f"类型：{first.book_type or '未填写'}")
        assert len(first.to_row()) == len(BOOK_COLUMNS)
    else:
        print("数据库目前没有书籍记录，空结果检查通过")

    print("3. 检查字典转换")
    demo = BookRecord.from_mapping({
        "book_name": "阶段0测试书",
        "author": "测试作者",
        "book_type": "测试类型",
    })
    assert demo.book_name == "阶段0测试书"
    assert demo.to_mapping()["author"] == "测试作者"
    print("字典转换检查通过")

    print("阶段 0 数据结构检查全部通过")
```

```
if __name__ == "__main__":
    main()
```

操作 15：运行验证脚本

```
cd D:\Eternity-Library
python verify_phase0.py
```

如果数据库尚未创建，脚本会提示数据库或表不存在。这不是数据结构代码错误；先运行一次原项目，让它创建 `books_information.db` 和 `books_information` 表，再重新运行验证脚本。

11. 第九步：按顺序启动项目验证

建议严格按下面顺序操作，每一步出现问题就先停下来解决，不要继续叠加修改。

1. 运行 `python -c "from book_record import BookRecord; print(BookRecord(book_name='ok'))"`。
2. 运行 `python verify_phase0.py`。
3. 启动 `python el_gui.py`，确认窗口能打开。
4. 确认表格模式的中文表头仍然显示。
5. 确认数据库有书籍时，封面模式可以显示书名、作者和类型。
6. 确认数据库为空时，界面不会保留上一次旧数据。
7. 扫描一小个测试目录，确认书籍仍能写入数据库。

出现错误时如何判断

错误现象	优先检查
`ModuleNotFoundError: book_record`	`book_record.py` 是否在项目根目录，运行命令是否位于 `D:\Eternity-Library`。
字段数量不是 19	数据库表是否是旧版本；查询是否仍然是 `SELECT *`；`CREATE TABLE` 是否被手动改过。
`BookRecord.from_row` 报字段数量错误	查询列数量和 `BOOK_COLUMNS` 不一致。检查是否只查询了部分列却仍调用 `from_row`。
表头数量或索引错误	`BOOK_COLUMNS` 与 `BOOK_COLUMN_TITLES` 是否一一对应。
封面模式报 `tuple` 或属性错误	`CoverCardModel.set_books` 是否已改成接收 `BookRecord`。
表格能显示但不能编辑	`BookTableModel.set_data` 是否使用 `list(book.to_row())`，`setData()` 是否仍修改内部列表。
扫描入库报未知字段	打印扫描字典的 keys，判断是拼写错误还是确实需要新增数据库字段。

12. 阶段 0 完成检查表

- □ 已新增 `book_record.py`，且没有把数据类写入 `database.py`。
- □ `BOOK_COLUMNS` 包含 19 个字段，顺序与数据库表一致。
- □ `BOOK_COLUMN_TITLES` 与字段数量一致。
- □ SQLite 查询结果可以转换为 `list[BookRecord]`。
- □ 错误不会再以字符串伪装成正常查询列表。
- □ 表格模型能把 `BookRecord` 转换成可编辑列表。
- □ 封面模型通过属性访问书名、作者和类型。
- □ 空查询结果会清空两个 Model。
- □ 扫描结果可以通过 `BookRecord.from_mapping()` 校验。
- □ GUI 可以启动，查询、表头和封面显示没有回归问题。
- □ 没有修改线程模型；阶段 1 再开始统一 DatabaseWorker。

13. 阶段 0 之后，下一步是什么

完成本教程后，不要立即删除旧的 `DatabaseManager` 或 `BookQueryWorker`。阶段 0 的目的只是让它们使用统一的数据结构。下一阶段才是：

1. 创建一个长期运行的 `DatabaseWorker`。
2. 把 SQLite 连接的创建和关闭放到 Worker 所在线程。
3. 让查询、写入、删除和更新都通过 Qt 信号槽请求。
4. 让 Worker 发出 `list[BookRecord]`，主线程更新 Model。

> **学习重点：**阶段 0 不是“为了增加一个文件而增加一个文件”，而是在建立项目中的共同语言。以后看到 `BookRecord`，就知道它代表一条完整书籍记录；看到 `list[BookRecord]`，就知道这是可以传递给界面 Model 的书籍列表。
